

Prompt 1. Validación del algoritmo

Estoy implementando en Python un árbol binario de expresión a partir de una expresión en notación postfija.

El algoritmo que comprendí es el siguiente:

1. Recorrer cada token.
2. Si es un operando, crear un nodo y colocarlo en una pila.
3. Si es un operador, extraer primero el hijo derecho y después el hijo izquierdo.
4. Crear un nodo operador, conectar sus hijos y colocarlo nuevamente en la pila.
5. Al terminar, el único nodo de la pila será la raíz.

Revisa únicamente si la lógica es correcta. No generes todavía el código. Indica posibles errores o casos que debo considerar.

Prompt 2. Apoyo para un método específico

Con base en el algoritmo validado, ayúdame a implementar únicamente el método `build_expression_tree`.

Debo utilizar la clase `Stack` que ya existe en mi proyecto. No utilices listas de Python como sustituto de la pila.

La clase del nodo se llama `NodeExpression` y tiene los atributos `value`, `left` y `right`.

Explica cada parte del método después del código.

1. Validación del algoritmo

La lógica que planteas es **correcta**. El procedimiento descrito es el estándar y más eficiente para construir un árbol de expresión a partir de una cadena en notación postfija (RPN).

Un detalle fundamental que has validado correctamente en el paso 3 es el **orden de extracción de la pila**: como la pila funciona bajo el principio LIFO (último en entrar, primero en salir), al encontrarnos con un operador, el primer elemento que extraemos corresponde al **hijo derecho** y el segundo al **hijo izquierdo**. Esto es exacto para respetar el orden de los operandos en expresiones como `A B -` (donde B es el operando derecho de la resta).

Posibles errores y casos a considerar:

- **Falta de operandos (Pila vacía):** Si la expresión está mal formada (por ejemplo, A +), al intentar hacer el segundo pop() para obtener el hijo izquierdo la pila estará vacía, lo que generará un error de desbordamiento (*stack underflow*).
- **Exceso de elementos al finalizar:** Si la expresión contiene operandos de más (por ejemplo, A B C +), al terminar el recorrido la pila tendrá más de un nodo en lugar de quedar únicamente con la raíz. Es recomendable validar que al final la pila tenga exactamente un elemento.
- **Identificación de tokens:** Debes asegurarte de tener un mecanismo claro para distinguir entre un operador y un operando (por ejemplo, mediante un conjunto de operadores válidos como {'+', '-', '*', '/'}).

2. Implementación del método build_expression_tree

A continuación se presenta la implementación del método cumpliendo con los requisitos de utilizar la clase Stack y la clase NodeExpression.

Python

```
def build_expression_tree(tokens, operadores_validos={'+', '-', '*', '/', '^'}):
```

```
    """
```

Construye un árbol binario de expresión a partir de una lista de tokens en postfijo.

Utiliza una instancia de la clase Stack existente en el proyecto.

```
    """
```

```
    pila = Stack()
```

```
    for token in tokens:
```

```
        # Verificar si el token es un operador
```

```
        if token in operadores_validos:
```

```
            # Extraer hijos respetando el orden LIFO de la pila
```

```
            right_node = pila.pop()
```

```
            left_node = pila.pop()
```

```

# Crear el nodo operador y conectar sus hijos
operator_node = NodeExpression(value=token)
operator_node.left = left_node
operator_node.right = right_node

# Colocar el subárbol resultante de vuelta en la pila
pila.push(operator_node)
else:
    # Si es un operando, crear un nodo hoja y colocarlo en la pila
    operand_node = NodeExpression(value=token)
    pila.push(operand_node)

# Al finalizar, el único elemento restante en la pila es la raíz del árbol
return pila.pop()

```

Explicación del código

- **Inicialización de la pila:** Se instancia el objeto Stack (`pila = Stack()`) que se utilizará exclusivamente para almacenar los nodos del árbol durante el proceso.
- **Recorrido de tokens (`for token in tokens`):** Se analiza cada elemento de la expresión postfija de izquierda a derecha.
- **Manejo de operadores:** Cuando el token es detectado dentro del conjunto de operadores_validos:
 1. Se extrae primero el nodo derecho (`pila.pop()`).
 2. Se extrae después el nodo izquierdo (`pila.pop()`).
 3. Se instancia un nuevo `NodeExpression` asignándole el operador como valor y enlazando los nodos hijos obtenidos.
 4. Se devuelve este subárbol a la pila mediante `pila.push()`.

- **Manejo de operandos:** Si el token no es un operador (se asume que es una variable o número), se encapsula directamente en un NodeExpression sin hijos y se apila.
- **Retorno del resultado:** Una vez procesados todos los tokens, la pila contendrá únicamente la raíz del árbol completo, la cual se extrae y retorna con `pila.pop()`.

Necesito que hagas un archivo parecido a este [from estructuras.no_lineales.binary_tree import BinaryTree

```
def mostrar_menu():
```

```
    print("\nÁRBOL BINARIO DE BÚSQUEDA")
```

```
    print("1. Insertar un valor")
```

```
    print("2. Buscar un valor")
```

```
    print("3. Recorrido en preorden")
```

```
    print("4. Recorrido en inorden")
```

```
    print("5. Recorrido en posorden")
```

```
    print("6. Contar nodo")
```

```
    print("7. Árbol de expresión")
```

```
    print("8. Salir")
```

```
tree = BinaryTree()
```

```
while True:
```

```
    mostrar_menu()
```

```
    option = input("Selecciona una opción: ")
```

```
    if option == "1":
```

```
        try:
```

```
            value = int(input("Ingresa el valor que deseas insertar: "))
```

```
        tree.insertar(value)
        print("Operación realizada.")
except ValueError:
    print("Debes ingresar un número entero.")

elif option == "2":
    try:
        value = int(input("Ingresa el valor que deseas buscar: "))
        if tree.buscar(value):
            print("El valor se encuentra en el árbol.")
        else:
            print("El valor no se encuentra en el árbol.")
    except ValueError:
        print("Debes ingresar un número entero.")

elif option == "3":
    print("Recorrido en preorden:")
    tree.preorden()

elif option == "4":
    print("Recorrido en inorden:")
    tree.inorden()

elif option == "5":
    print("Recorrido en posorden:")
    tree.posorden()
```

```

elif option == "6":
    print(tree.contar())

elif option == "7":
    print (tree.build_expression_tree(input("Ingresa la expresión postfija: ")))
elif option == "8":
    print("Programa finalizado.")
    break
else:
    print("Opción no válida. Intenta nuevamente.")

```

] pero con el arbol de expresion por favor

Aquí tienes el archivo de menú interactivo adaptado específicamente para trabajar con el **Árbol de Expresión**, integrando la lógica que vimos anteriormente (separando tokens mediante espacios, utilizando tu estructura de clases y mostrando los recorridos típicos del árbol de expresión como prefija, infija y postfija).

Python

```

from estructuras.no_lineales.expression_tree import ExpressionTree

```

```

def mostrar_menu():
    print("\nÁRBOL DE EXPRESIÓN")
    print("1. Ingresar expresión en notación postfija")
    print("2. Recorrido en Preorden (Prefija)")
    print("3. Recorrido en Inorden (Infija)")
    print("4. Recorrido en Posorden (Postfija original)")
    print("5. Evaluar expresión")
    print("6. Salir")

```

```
tree = ExpressionTree()
```

```
while True:
```

```
    mostrar_menu()
```

```
    option = input("Selecciona una opción: ")
```

```
    if option == "1":
```

```
        expresion = input("Ingresa la expresión postfija (separa los tokens con  
espacios, ej: A B + C *): ")
```

```
        tokens = expresion.split()
```

```
        if tokens:
```

```
            try:
```

```
                tree.build_expression_tree(tokens)
```

```
                print("Árbol de expresión construido exitosamente.")
```

```
            except Exception as e:
```

```
                print(f"Error al construir el árbol: {e}")
```

```
        else:
```

```
            print("La expresión no puede estar vacía.")
```

```
    elif option == "2":
```

```
        print("Recorrido en preorden (Prefija):")
```

```
        try:
```

```
            tree.preorden()
```

```
        except Exception as e:
```

```
            print(f"Primero debes construir el árbol. Error: {e}")
```

```
    elif option == "3":
```

```

print("Recorrido en inorden (Infija):")
try:
    tree.inorden()
except Exception as e:
    print(f"Primero debes construir el árbol. Error: {e}")

elif option == "4":
    print("Recorrido en posorden (Postfija):")
    try:
        tree.posorden()
    except Exception as e:
        print(f"Primero debes construir el árbol. Error: {e}")

elif option == "5":
    print("Evaluación de la expresión:")
    try:
        # Opcional: si tu clase ya implementa un método para evaluar con
        valores numéricos
        resultado = tree.evaluar()
        print(f"Resultado: {resultado}")
    except AttributeError:
        print("El método de evaluación no está implementado en la clase
        ExpressionTree.")
    except Exception as e:
        print(f"Error al evaluar: {e}")

elif option == "6":
    print("Programa finalizado.")

```



```
break
```

```
else:
```

```
    print("Opción no válida. Intenta nuevamente.")
```

Notas importantes para que encaje con tu proyecto:

1. **Importación:** Asegúrate de que la ruta de importación (from estructuras.no_lineales.expression_tree import ExpressionTree) coincida con la ubicación y nombre real del archivo donde guardaste tu clase del árbol de expresión.
2. **Tokens por espacios:** Se agregó .split() en la opción 1 para que al ingresar la expresión (por ejemplo: 3 4 + 2 *), Python la reconozca como una lista de tokens separados (['3', '4', '+', '2', '*']), lo cual es necesario para que el algoritmo procese correctamente operadores y operandos de más de un carácter o dígito.

Ingresa la expresión postfija (separa los tokens con espacios, ej: A B + C *): 3 + 4

Error al construir el árbol: 'NodeExpression' object has no attribute 'build_expression_tree'

El error ocurre porque estás intentando llamar al método build_expression_tree directamente sobre la variable tree, pero tree parece ser una instancia de NodeExpression (o la función está definida dentro de otra clase o módulo), la cual no cuenta con ese método.

Para solucionarlo, asegúrate de que tu estructura cuente con una clase contenedora (por ejemplo, ExpressionTree) que gestione la raíz y el método de construcción utilizando la pila (Stack).

A continuación tienes el código corregido de la clase y cómo debe estructurarse para que el menú funcione correctamente:

1. Estructura de clases recomendada

Asegúrate de tener un archivo (por ejemplo, en estructuras/no_lineales/expression_tree.py) con una estructura similar a esta:

Python

```
class NodeExpression:
```

```
    def __init__(self, value):
```

```
self.value = value
```

```
self.left = None
```

```
self.right = None
```

```
class ExpressionTree:
```

```
def __init__(self):
```

```
    self.root = None
```

```
def build_expression_tree(self, tokens):
```

```
    pila = Stack() # Asegúrate de importar tu clase Stack aquí
```

```
    operadores_validos = {'+', '-', '*', '/', '^'}
```

```
    for token in tokens:
```

```
        if token in operadores_validos:
```

```
            right_node = pila.pop()
```

```
            left_node = pila.pop()
```

```
            operator_node = NodeExpression(value=token)
```

```
            operator_node.left = left_node
```

```
            operator_node.right = right_node
```

```
            pila.push(operator_node)
```

```
        else:
```

```
            operand_node = NodeExpression(value=token)
```

```
            pila.push(operand_node)
```

```
self.root = pila.pop()
```

2. Archivo del Menú Actualizado

Una vez que tu clase ExpressionTree contenga el método correctamente, este script de menú funcionará sin errores:

Python

```
from estructuras.no_lineales.expression_tree import ExpressionTree
```

```
def mostrar_menu():
```

```
    print("\nÁRBOL DE EXPRESIÓN")
```

```
    print("1. Ingresar expresión en notación postfija")
```

```
    print("2. Recorrido en Preorden (Prefija)")
```

```
    print("3. Recorrido en Inorden (Infija)")
```

```
    print("4. Recorrido en Posorden (Postfija original)")
```

```
    print("5. Salir")
```

```
tree = ExpressionTree()
```

```
while True:
```

```
    mostrar_menu()
```

```
    option = input("Selecciona una opción: ")
```

```
    if option == "1":
```

```
        expresion = input("Ingresa la expresión postfija (separa los tokens con espacios, ej: 3 4 +): ")
```

```
        tokens = expresion.split()
```

```
        if tokens:
```

```
            try:
```

```
                tree.build_expression_tree(tokens)
```

```
                print("Árbol de expresión construido exitosamente.")
```

```

except Exception as e:
    print(f"Error al construir el árbol (revisa si la expresión está en postfija
válida): {e}")
else:
    print("La expresión no puede estar vacía.")

elif option == "2":
    print("Recorrido en preorden no implementado aún.")

elif option == "3":
    print("Recorrido en inorden no implementado aún.")

elif option == "4":
    print("Recorrido en posorden no implementado aún.")

elif option == "5":
    print("Programa finalizado.")
    break
else:
    print("Opción no válida. Intenta nuevamente.")

```

Nota sobre tu ejemplo: En el menú ingresaste $3 + 4$, lo cual está en notación **infija** (con el operador en medio). Recuerda que el algoritmo que validamos y construimos requiere notación **postfija** (por ejemplo, ingresando $3\ 4\ +$).

corrige estos codigos from estructuras.no_lineales.expression_tree import
NodeExpression

```

def mostrar_menu():
    print("\nÁRBOL DE EXPRESIÓN")

```

```
print("1. Ingresar expresión en notación postfija")
print("2. Recorrido en Preorden (Prefija)")
print("3. Recorrido en Inorden (Infija)")
print("4. Recorrido en Posorden (Postfija original)")
print("5. Evaluar expresión")
print("6. Salir")
```

```
tree = NodeExpression(None) # Inicializa el árbol de expresión vacío
```

```
while True:
```

```
    mostrar_menu()
```

```
    option = input("Selecciona una opción: ")
```

```
    if option == "1":
```

```
        expresion = input("Ingresa la expresión postfija (separa los tokens con  
espacios, ej: A B + C *): ")
```

```
        tokens = expresion.split()
```

```
        if tokens:
```

```
            try:
```

```
                tree.build_expression_tree(tokens)
```

```
                print("Árbol de expresión construido exitosamente.")
```

```
            except Exception as e:
```

```
                print(f"Error al construir el árbol: {e}")
```

```
        else:
```

```
            print("La expresión no puede estar vacía.")
```

```
    elif option == "2":
```

```

print("Recorrido en preorden (Prefija):")
try:
    tree.preorden()
except Exception as e:
    print(f"Primero debes construir el árbol. Error: {e}")

elif option == "3":
    print("Recorrido en inorden (Infija):")
    try:
        tree.inorden()
    except Exception as e:
        print(f"Primero debes construir el árbol. Error: {e}")

elif option == "4":
    print("Recorrido en posorden (Postfija):")
    try:
        tree.posorden()
    except Exception as e:
        print(f"Primero debes construir el árbol. Error: {e}")

elif option == "5":
    print("Evaluación de la expresión:")
    try:
        # Opcional: si tu clase ya implementa un método para evaluar con
        valores numéricos
        resultado = tree.evaluar()
        print(f"Resultado: {resultado}")

```

```

except AttributeError:

    print("El método de evaluación no está implementado en la clase
ExpressionTree.")

except Exception as e:

    print(f"Error al evaluar: {e}")


elif option == "6":

    print("Programa finalizado.")

    break

else:

    print("Opción no válida. Intenta nuevamente.") y from
estructuras.lineales.stack import Stack

class NodeExpression:

    def __init__(self, value):

        self.value = value # valor almacenado

        self.left = None # se inician sin hijos

        self.right = None


def build_expression_tree(tokens, operadores_validos={'+', '-', '*', '/', '^'}):

    """

    Construye un árbol binario de expresión a partir de una lista de tokens en
    postfijo.

    Utiliza una instancia de la clase Stack existente en el proyecto.

    """

    pila = Stack()


    for token in tokens:

        # Verificar si el token es un operador

```

```

if token in operadores_validos:

    # Extraer hijos respetando el orden LIFO de la pila
    right_node = pila.pop()
    left_node = pila.pop()

    # Crear el nodo operador y conectar sus hijos
    operator_node = NodeExpression(value=token)
    operator_node.left = left_node
    operator_node.right = right_node

    # Colocar el subárbol resultante de vuelta en la pila
    pila.push(operator_node)
else:

    # Si es un operando, crear un nodo hoja y colocarlo en la pila
    operand_node = NodeExpression(value=token)
    pila.push(operand_node)

# Al finalizar, el único elemento restante en la pila es la raíz del árbol
return pila.pop()

```

Para que tus códigos funcionen correctamente en conjunto, debemos solucionar un detalle de diseño: **NodeExpression es un nodo individual, no el árbol completo**. Por lo tanto, no puede contener métodos como `build_expression_tree()` o los recorridos por sí mismo.

La solución correcta es crear una clase contenedora llamada **ExpressionTree** que administre la raíz (`self.root`) y mueva los métodos de recorrido y construcción hacia ella, conectándolos con tu clase `Stack`.

A continuación tienes los dos códigos corregidos y listos para integrarse:

1. Archivo de la Estructura (estructuras/no_lineales/expression_tree.py)

Python


```
from estructuras.lineales.stack import Stack
```

```
class NodeExpression:
```

```
    def __init__(self, value):  
        self.value = value # valor almacenado  
        self.left = None # se inician sin hijos  
        self.right = None
```

```
class ExpressionTree:
```

```
    def __init__(self):  
        self.root = None
```

```
    def build_expression_tree(self, tokens, operadores_validos={'+', '-', '*', '/', '^}):
```

```
        """
```

```
        Construye el árbol de expresión a partir de una lista de tokens en postfijo  
        y almacena la raíz en self.root.
```

```
        """
```

```
        pila = Stack()
```

```
        for token in tokens:
```

```
            if token in operadores_validos:
```

```
                right_node = pila.pop()
```

```
                left_node = pila.pop()
```

```
                operator_node = NodeExpression(value=token)
```

```
                operator_node.left = left_node
```

```
                operator_node.right = right_node
```

```

        pila.push(operator_node)
    else:
        operand_node = NodeExpression(value=token)
        pila.push(operand_node)

self.root = pila.pop()

# Métodos de recorrido recursivos
def preorden(self):
    def _preorden(node):
        if node:
            print(node.value, end=" ")
            _preorden(node.left)
            _preorden(node.right)

    if not self.root:
        raise Exception("El árbol está vacío.")
    _preorden(self.root)
    print()

def inorden(self):
    def _inorden(node):
        if node:
            _inorden(node.left)
            print(node.value, end=" ")
            _inorden(node.right)

```

```
if not self.root:
    raise Exception("El árbol está vacío.")
_inorden(self.root)
print()
```

```
def posorden(self):
    def _posorden(node):
        if node:
            _posorden(node.left)
            _posorden(node.right)
            print(node.value, end=" ")
```

```
if not self.root:
    raise Exception("El árbol está vacío.")
_posorden(self.root)
print()
```

2. Archivo del Menú Principal (Main)

Python

```
from estructuras.no_lineales.expression_tree import ExpressionTree
```

```
def mostrar_menu():
    print("\nÁRBOL DE EXPRESIÓN")
    print("1. Ingresar expresión en notación postfija")
    print("2. Recorrido en Preorden (Prefija)")
    print("3. Recorrido en Inorden (Infija)")
    print("4. Recorrido en Posorden (Postfija original)")
```

```
print("5. Evaluar expresión")
```

```
print("6. Salir")
```

```
tree = ExpressionTree() # Inicializa el contenedor del árbol de expresión
```

```
while True:
```

```
    mostrar_menu()
```

```
    option = input("Selecciona una opción: ")
```

```
    if option == "1":
```

```
        expresion = input("Ingresa la expresión postfija (separa los tokens con  
espacios, ej: A B + C *): ")
```

```
        tokens = expresion.split()
```

```
        if tokens:
```

```
            try:
```

```
                tree.build_expression_tree(tokens)
```

```
                print("Árbol de expresión construido exitosamente.")
```

```
            except Exception as e:
```

```
                print(f"Error al construir el árbol: {e}")
```

```
        else:
```

```
            print("La expresión no puede estar vacía.")
```

```
    elif option == "2":
```

```
        print("Recorrido en preorden (Prefija):")
```

```
        try:
```

```
            tree.preorden()
```

```
        except Exception as e:
```

```
print(f"Primero debes construir el árbol. Error: {e}")
```

```
elif option == "3":
```

```
    print("Recorrido en inorden (Infija):")
```

```
    try:
```

```
        tree.inorden()
```

```
    except Exception as e:
```

```
        print(f"Primero debes construir el árbol. Error: {e}")
```

```
elif option == "4":
```

```
    print("Recorrido en posorden (Postfija):")
```

```
    try:
```

```
        tree.posorden()
```

```
    except Exception as e:
```

```
        print(f"Primero debes construir el árbol. Error: {e}")
```

```
elif option == "5":
```

```
    print("Evaluación de la expresión:")
```

```
    try:
```

```
        resultado = tree.evaluar()
```

```
        print(f"Resultado: {resultado}")
```

```
    except AttributeError:
```

```
        print("El método de evaluación no está implementado en la clase  
ExpressionTree.")
```

```
    except Exception as e:
```

```
        print(f"Error al evaluar: {e}")
```

```

elif option == "6":
    print("Programa finalizado.")
    break
else:
    print("Opción no válida. Intenta nuevamente.")

```

Ahora necesito una librería en donde en la consola aparezca el árbol construido

Para mostrar el árbol de expresión gráficamente en la consola de manera estructurada y limpia, la mejor alternativa es utilizar la librería externa **asciitree** o implementar un algoritmo de impresión jerárquica personalizado.

Como en proyectos escolares o modulares a veces no se pueden instalar librerías externas fácilmente mediante pip, te ofrezco **dos opciones**:

Opción 1: Método personalizado (Sin instalar librerías externas)

Puedes agregar este método directamente dentro de tu clase `ExpressionTree` (en `expression_tree.py`). Utiliza una técnica de recorrido recursivo invertido para dibujar el árbol de lado, lo cual permite visualizar perfectamente las ramas izquierda y derecha sin importar qué tan grande sea el árbol.

Agrega este método a tu clase `ExpressionTree`:

Python

```

def imprimir_arbol_visual(self):
    """Imprime una representación visual del árbol en la consola."""
    def _mostrar(nodo, prefix="", is_left=True):
        if nodo is not None:
            print(prefix + (" |—" if is_left else " L— ") + str(nodo.value))
            # Llamadas recursivas para los hijos (izquierdo y derecho)
            _mostrar(nodo.left, prefix + (" | " if is_left else "   "), True)
            _mostrar(nodo.right, prefix + (" | " if is_left else "   "), False)

    if not self.root:

```

```
print("El árbol está vacío.")
```

```
return
```

```
print(str(self.root.value))
```

```
_mostrar(self.root.left, "", True)
```

```
_mostrar(self.root.right, "", False)
```

Opción 2: Usar una librería externa (asciitree o treelib)

Si tienes la libertad de instalar librerías mediante la terminal (pip install), la librería **treelib** es excelente para esto.

1. Instalación:

Bash

```
pip install treelib
```

2. **Método adaptador para tu clase:** Puedes agregar este método a tu ExpressionTree para convertir tus nodos a la estructura que treelib entiende y mostrarlo automáticamente:

Python

```
from treelib import Tree
```

```
def imprimir_con_treelib(self):
```

```
    if not self.root:
```

```
        print("El árbol está vacío.")
```

```
        return
```

```
    t = Tree()
```

```
    # Función auxiliar recursiva para poblar el árbol visual
```

```
    def _agregar_nodos(nodo, padre_id=None):
```

```
        if nodo is not None:
```

```
# Generamos un identificador único basado en el objeto para evitar
colisiones si hay valores repetidos
```

```
node_id = str(id(nodo))
```

```
t.create_node(str(nodo.value), node_id, parent=padre_id)
```

```
if nodo.left:
```

```
    _agregar_nodos(nodo.left, node_id)
```

```
if nodo.right:
```

```
    _agregar_nodos(nodo.right, node_id)
```

```
t.create_node(str(self.root.value), str(id(self.root)))
```

```
if self.root.left:
```

```
    _agregar_nodos(self.root.left, str(id(self.root)))
```

```
if self.root.right:
```

```
    _agregar_nodos(self.root.right, str(id(self.root)))
```

```
t.show()
```

Integración en tu menú principal

Puedes agregar una nueva opción (por ejemplo, la **opción 6**) en tu menú para llamar a esta función visual:

Python

```
elif option == "6":
```

```
    print("\nRepresentación gráfica del árbol:")
```

```
    try:
```

```
        tree.imprimir_arbol_visual() # O tree.imprimir_con_treelib()
```

```
    except Exception as e:
```

```
        print(f"Error al mostrar el árbol: {e}")
```


Ahora necesito pasar a interfaz en Qt Designer. Necesito que me hagas el archivo .ui en donde se le pida al usuario la operación y este lo convierta automáticamente en todas las demás operaciones y que automáticamente también se muestre el gráfico del árbol de expresión, gráficamente en grande, que se tiene que ver, por favor.

Para crear una interfaz gráfica profesional en **Qt Designer** que cumpla con todos tus requisitos (ingresar la expresión, convertirla automáticamente a todas las demás notaciones, ver los resultados y desplegar el gráfico grande del árbol), el flujo ideal es diseñar un archivo .ui con una distribución limpia y moderna.

Como Qt Designer genera un archivo XML (.ui), a continuación te presento **el código fuente completo del archivo XML** que puedes guardar directamente con el nombre expression_tree_window.ui, junto con el código en Python que conecta dicha interfaz con tus clases (ExpressionTree, Stack, etc.) y genera el gráfico grande.

1. Código del archivo expression_tree_window.ui

Crea un archivo de texto en tu proyecto, cópiale este contenido y guárdalo como expression_tree_window.ui:

XML

```
<?xml version="1.0" encoding="UTF-8"?>
<ui version="4.0">
  <class>ExpressionTreeWindow</class>
  <widget class="QMainWindow" name="ExpressionTreeWindow">
    <property name="geometry">
      <rect>
        <x>0</x>
        <y>0</y>
        <width>900</width>
        <height>700</height>
      </rect>
    </property>
```

```
<property name="windowTitle">
  <string>Visualizador de Árboles de Expresión</string>
</property>
<widget class="QWidget" name="centralwidget">
  <layout class="QVBoxLayout" name="verticalLayout_main">
    <item>
      <layout class="QHBoxLayout" name="horizontalLayout_input">
        <item>
          <widget class="QLabel" name="label_input">
            <property name="font">
              <font>
                <pointsize>11</pointsize>
                <bold>true</bold>
              </font>
            </property>
            <property name="text">
              <string>Expresión Postfija:</string>
            </property>
          </widget>
        </item>
        <item>
          <widget class="QLineEdit" name="lineEdit_expresion">
            <property name="font">
              <font>
                <pointsize>11</pointsize>
              </font>
            </property>
```

```
<property name="placeholderText">
  <string>Ej: A B + C * (separa con espacios)</string>
</property>
</widget>
</item>
<item>
  <widget class="QPushButton" name="btn_procesar">
    <property name="font">
      <font>
        <pointsize>11</pointsize>
        <bold>true</bold>
      </font>
    </property>
    <property name="text">
      <string>Procesar y Convertir</string>
    </property>
  </widget>
</item>
</layout>
</item>
<item>
  <layout class="QGridLayout" name="gridLayout_outputs">
    <item row="0" column="0">
      <widget class="QLabel" name="label_pre">
        <property name="text">
          <string>Preorden (Prefija):</string>
        </property>
```

```
</widget>
</item>
<item row="0" column="1">
  <widget class="QLineEdit" name="lineEdit_pre">
    <property name="readOnly">
      <bool>true</bool>
    </property>
  </widget>
</item>
<item row="1" column="0">
  <widget class="QLabel" name="label_in">
    <property name="text">
      <string>Inorden (Infija):</string>
    </property>
  </widget>
</item>
<item row="1" column="1">
  <widget class="QLineEdit" name="lineEdit_in">
    <property name="readOnly">
      <bool>true</bool>
    </property>
  </widget>
</item>
<item row="2" column="0">
  <widget class="QLabel" name="label_post">
    <property name="text">
      <string>Posorden (Postfija):</string>
```

```
</property>
</widget>
</item>
<item row="2" column="1">
  <widget class="QLineEdit" name="lineEdit_post">
    <property name="readOnly">
      <bool>true</bool>
    </property>
  </widget>
</item>
</layout>
</item>
<item>
  <widget class="QGroupBox" name="groupBox_grafico">
    <property name="font">
      <font>
        <pointsize>11</pointsize>
        <bold>true</bold>
      </font>
    </property>
    <property name="title">
      <string>Gráfico del Árbol de Expresión</string>
    </property>
    <layout class="QVBoxLayout" name="verticalLayout_grafico">
      <item>
        <widget class="QGraphicsView" name="graphicsView_tree"/>
      </item>
    </layout>
  </widget>
</item>
```

```

    </layout>
  </groupBox>
</item>
</layout>
</widget>
<widget class="QMenuBar" name="menubar">
  <property name="geometry">
    <rect>
      <x>0</x>
      <y>0</y>
      <width>900</width>
      <height>22</height>
    </rect>
  </property>
</widget>
<widget class="QStatusBar" name="statusbar"/>
</widget>
<resources/>
<connections/>
</ui>

```

2. Controlador en Python (main_gui.py)

Para que el gráfico del árbol se dibuje **en grande y de forma visualmente limpia**, utilizaremos QGraphicsScene y QGraphicsView de PyQt, calculando de manera automática las coordenadas de los nodos en forma de nodos circulares con sus respectivas líneas conectoras.

Crea un archivo controlador (ej. main.py o main_gui.py) que cargue el .ui anterior y conecte tu clase ExpressionTree:

Python

```

import sys

from PyQt5.QtWidgets import QApplication, QMainWindow, QMessageBox

from PyQt5.uic import loadUi

from PyQt5.QtGui import QPen, QBrush, QFont

from PyQt5.QtCore import Qt, QRectF


# Importa tus clases existentes

from estructuras.no_lineales.expression_tree import ExpressionTree


class ExpressionTreeGUI(QMainWindow):

    def __init__(self):
        super().__init__()

        # Carga el archivo .ui que creamos
        loadUi("expression_tree_window.ui", self)

        self.tree = ExpressionTree()

        # Conectar el botón al evento de procesamiento automático
        self.btn_procesar.clicked.connect(self.procesar_expresion)
        self.lineEdit_expresion.returnPressed.connect(self.procesar_expresion)

    def procesar_expresion(self):
        texto = self.lineEdit_expresion.text().strip()

        if not texto:
            QMessageBox.warning(self, "Advertencia", "Por favor ingresa una
            expresión postfija.")

        return

```

```

tokens = texto.split()

try:

    # 1. Construir el árbol con tu lógica
    self.tree.build_expression_tree(tokens)

    # 2. Generar automáticamente todas las conversiones en los campos
de texto

    self.lineEdit_pre.setText(self.obtener_recorrido_preorden())
    self.lineEdit_in.setText(self.obtener_recorrido_inorden())
    self.lineEdit_post.setText(self.obtener_recorrido_postorden())

    # 3. Dibujar automáticamente el árbol en grande dentro del
QGraphicsView

    self.dibujar_arbol_grafico()

except Exception as e:

    QMessageBox.critical(self, "Error", f'Ocurrió un error al procesar la
expresión:\n{str(e)}')

# Métodos auxiliares para capturar los strings de los recorridos
def obtener_recorrido_preorden(self):
    resultado = []
    def _pre(node):
        if node:
            resultado.append(str(node.value))
            _pre(node.left)
            _pre(node.right)

```



```
_pre(self.tree.root)
return " ".join(resultado)
```

```
def obtener_recorrido_inorden(self):
    resultado = []
    def _in(node):
        if node:
            _in(node.left)
            resultado.append(str(node.value))
            _in(node.right)
    _in(self.tree.root)
    return " ".join(resultado)
```

```
def obtener_recorrido_postorden(self):
    resultado = []
    def _post(node):
        if node:
            _post(node.left)
            _post(node.right)
            resultado.append(str(node.value))
    _post(self.tree.root)
    return " ".join(resultado)
```

```
def dibujar_arbol_grafico(self):
    from PyQt5.QtWidgets import QGraphicsScene

    scene = QGraphicsScene()
```

```

self.graphicsView_tree.setScene(scene)

if not self.tree.root:
    return

# Parámetros visuales para nodos grandes y claros
radio_nodo = 25
nivel_altura = 70

# Algoritmo para calcular posiciones (Coordenadas X, Y) basadas en un
recorrido recursivo
def calcular_posiciones(node, nivel=0, x_min=0, x_max=800):
    if not node:
        return {}

    posiciones = {}
    x_actual = (x_min + x_max) / 2
    y_actual = 40 + (nivel * nivel_altura)
    posiciones[node] = (x_actual, y_actual)

    # Dividir el espacio horizontal para los hijos izquierdo y derecho
    if node.left:
        posiciones.update(calcular_posiciones(node.left, nivel + 1, x_min,
x_actual))
    if node.right:
        posiciones.update(calcular_posiciones(node.right, nivel + 1, x_actual,
x_max))

```

```
return posiciones
```

```
pos_dict = calcular_posiciones(self.tree.root, nivel=0, x_min=0,  
x_max=900)
```

```
# Dibujar líneas conectoras (aristas) primero para que queden por debajo  
de los nodos
```

```
pen_linea = QPen(Qt.black, 2)
```

```
def dibujar_conexiones(node):
```

```
    if not node:
```

```
        return
```

```
    x1, y1 = pos_dict[node]
```

```
    if node.left:
```

```
        x2, y2 = pos_dict[node.left]
```

```
        scene.addLine(x1, y1, x2, y2, pen_linea)
```

```
        dibujar_conexiones(node.left)
```

```
    if node.right:
```

```
        x2, y2 = pos_dict[node.right]
```

```
        scene.addLine(x1, y1, x2, y2, pen_linea)
```

```
        dibujar_conexiones(node.right)
```

```
dibujar_conexiones(self.tree.root)
```

```
# Dibujar los nodos en grande (círculos con texto centrado)
```

```
brush_nodo = QBrush(Qt.lightGray)
```

```
pen_nodo = QPen(Qt.darkBlue, 2)
```

```
font_nodo = QFont("Arial", 11, QFont.Bold)
```

```

for node, (x, y) in pos_dict.items():

    # Círculo del nodo

    scene.addEllipse(x - radio_nodo, y - radio_nodo, radio_nodo * 2,
radio_nodo * 2, pen_nodo, brush_nodo)


    # Texto dentro del nodo

    text_item = scene.addText(str(node.value), font_nodo)

    text_rect = text_item.boundingRect()

    text_item.setPos(x - text_rect.width() / 2, y - text_rect.height() / 2)


if __name__ == "__main__":
    app = QApplication(sys.argv)
    ventana = ExpressionTreeGUI()
    ventana.show()
    sys.exit(app.exec_())

```

Instrucciones de uso:

1. Asegúrate de tener instalada la librería PyQt5 (pip install PyQt5).
2. Guarda el bloque XML con el nombre `expression_tree_window.ui` en la misma carpeta donde tienes tu proyecto de Python.
3. Ejecuta el script de Python. Al escribir una expresión en postfija (por ejemplo, `A B + C * D /`) y hacer clic en **"Procesar y Convertir"**, la interfaz automáticamente llenará los campos de **Preorden, Inorden y Posorden**, y renderizará un diagrama grande y estructurado del árbol de expresión en el visor gráfico inferior.

Necesito que al código del archivo de UI ya esté con los requerimientos que te pedí y que tenga un diseño muy profesional.

Para lograr un **diseño completamente profesional y moderno**, he actualizado el código del archivo `.ui` aplicando una interfaz inspirada en los estándares actuales de software (estilo minimalista, paleta de colores corporativa con tonos azul oscuro/gris pizarra, tipografías limpias y una distribución simétrica).

Además, para asegurarme de que el archivo .ui cumpla al 100% con todos tus requerimientos visuales y funcionales sin que tengas que modificar nada por fuera, incluye directamente la estructura de diseño optimizada.

1. Archivo `expression_tree_window.ui` (Diseño Profesional)

Copia y reemplaza el contenido de tu archivo `expression_tree_window.ui` con el siguiente código XML:

XML

```
<?xml version="1.0" encoding="UTF-8"?>

<ui version="4.0">

    <class>ExpressionTreeWindow</class>

    <widget class="QMainWindow" name="ExpressionTreeWindow">

        <property name="geometry">
            <rect>
                <x>0</x>
                <y>0</y>
                <width>1050</width>
                <height>750</height>
            </rect>
        </property>

        <property name="windowTitle">
            <string>Analizador y Visualizador de Árboles de Expresión</string>
        </property>

        <property name="styleSheet">
            <string notr="true">

                QMainWindow {
                    background-color: #f4f6f9;

                }

                QLabel {
```

```
    color: #2c3e50;
    font-family: 'Segoe UI', Arial, sans-serif;
}

QLineEdit {
    background-color: #ffffff;
    border: 2px solid #cbd5e1;
    border-radius: 6px;
    padding: 6px 10px;
    color: #1e293b;
    font-family: 'Segoe UI', Arial, sans-serif;
}

QLineEdit:focus {
    border: 2px solid #3b82f6;
}

QLineEdit[readOnly="true"] {
    background-color: #f1f5f9;
    color: #475569;
}

QPushButton {
    background-color: #2563eb;
    color: #ffffff;
    border-radius: 6px;
    padding: 8px 16px;
    font-weight: bold;
    font-family: 'Segoe UI', Arial, sans-serif;
}

QPushButton:hover {
```

```
        background-color: #1d4ed8;
    }
    QPushButton:pressed {
        background-color: #1e40af;
    }
    QGroupBox {
        font-family: 'Segoe UI', Arial, sans-serif;
        font-weight: bold;
        color: #1e293b;
        border: 2px solid #e2e8f0;
        border-radius: 8px;
        margin-top: 12px;
        background-color: #ffffff;
    }
    QGroupBox::title {
        subcontrol-origin: margin;
        subcontrol-position: top left;
        padding: 0 8px;
        background-color: #f4f6f9;
        color: #1e293b;
    }
    QGraphicsView {
        background-color: #ffffff;
        border: 1px solid #e2e8f0;
        border-radius: 6px;
    }
</string>
```

```
</property>
<widget class="QWidget" name="centralwidget">
  <layout class="QVBoxLayout" name="verticalLayout_main">
    <property name="spacing">
      <number>16</number>
    </property>
    <property name="leftMargin">
      <number>20</number>
    </property>
    <property name="topMargin">
      <number>20</number>
    </property>
    <property name="rightMargin">
      <number>20</number>
    </property>
    <property name="bottomMargin">
      <number>20</number>
    </property>

  <!-- SECCIÓN DE ENTRADA -->
  <item>
    <widget class="QGroupBox" name="groupBox_input">
      <property name="title">
        <string> Entrada de Datos </string>
      </property>
      <layout class="QHBoxLayout" name="horizontalLayout_input">
        <property name="spacing">
```



```
<number>12</number>
</property>
<item>
  <widget class="QLabel" name="label_input">
    <property name="font">
      <font>
        <pointsize>11</pointsize>
        <bold>true</bold>
      </font>
    </property>
    <property name="text">
      <string>Expresión Postfija:</string>
    </property>
  </widget>
</item>
<item>
  <widget class="QLineEdit" name="lineEdit_expresion">
    <property name="font">
      <font>
        <pointsize>11</pointsize>
      </font>
    </property>
    <property name="placeholderText">
      <string>Ej: A B + C * D / (separa los tokens con espacios)</string>
    </property>
  </widget>
</item>
```

```

<item>
  <widget class="QPushButton" name="btn_procesar">
    <property name="font">
      <font>
        <pointsize>11</pointsize>
        <bold>true</bold>
      </font>
    </property>
    <property name="cursor">
      <cursorShape>PointingHandCursor</cursorShape>
    </property>
    <property name="text">
      <string>Generar Árbol y Conversiones</string>
    </property>
  </widget>
</item>
</layout>
</widget>
</item>

<!-- SECCIÓN DE RESULTADOS / CONVERSIONES -->
<item>
  <widget class="QGroupBox" name="groupBox_conversiones">
    <property name="title">
      <string> Conversiones Automáticas de Notación </string>
    </property>
    <layout class="QGridLayout" name="gridLayout_outputs">

```

```
<property name="horizontalSpacing">
  <number>15</number>
</property>
<property name="verticalSpacing">
  <number>10</number>
</property>
<item row="0" column="0">
  <widget class="QLabel" name="label_pre">
    <property name="font">
      <font>
        <pointsize>10</pointsize>
        <bold>true</bold>
      </font>
    </property>
    <property name="text">
      <string>Preorden (Prefija):</string>
    </property>
  </widget>
</item>
<item row="0" column="1">
  <widget class="QLineEdit" name="lineEdit_pre">
    <property name="font">
      <font>
        <pointsize>10</pointsize>
      </font>
    </property>
    <property name="readOnly">
```

```
<bool>true</bool>

</property>

</widget>

</item>

<item row="1" column="0">

  <widget class="QLabel" name="label_in">

    <property name="font">

      <font>

        <pointsize>10</pointsize>

        <bold>true</bold>

      </font>

    </property>

    <property name="text">

      <string>Inorden (Infija):</string>

    </property>

  </widget>

</item>

<item row="1" column="1">

  <widget class="QLineEdit" name="lineEdit_in">

    <property name="font">

      <font>

        <pointsize>10</pointsize>

      </font>

    </property>

    <property name="readOnly">

      <bool>true</bool>

    </property>
```

```
</widget>
</item>
<item row="2" column="0">
  <widget class="QLabel" name="label_post">
    <property name="font">
      <font>
        <pointsize>10</pointsize>
        <bold>true</bold>
      </font>
    </property>
    <property name="text">
      <string>Posorden (Postfija):</string>
    </property>
  </widget>
</item>
<item row="2" column="1">
  <widget class="QLineEdit" name="lineEdit_post">
    <property name="font">
      <font>
        <pointsize>10</pointsize>
      </font>
    </property>
    <property name="readOnly">
      <bool>true</bool>
    </property>
  </widget>
</item>
```

</layout>

</widget>

</item>

<!-- SECCIÓN DEL GRÁFICO DEL ÁRBOL -->

<item>

<widget class="QGroupBox" name="groupBox_grafico">

<property name="title">

<string> Representación Gráfica del Árbol de Expresión </string>

</property>

<layout class="QVBoxLayout" name="verticalLayout_grafico">

<item>

<widget class="QGraphicsView" name="graphicsView_tree">

<property name="renderHints">

<set>QPainter::Antialiasing|QPainter::TextAntialiasing</set>

</property>

</widget>

</item>

</layout>

</widget>

</item>

</layout>

</widget>

<widget class="QMenuBar" name="menubar">

<property name="geometry">

<rect>

<x>0</x>

```
<y>0</y>
<width>1050</width>
<height>22</height>
</rect>
</property>
</widget>
<widget class="QStatusBar" name="statusbar"/>
</widget>
<resources/>
<connections/>
</ui>
```

Mejoras de diseño profesional incorporadas:

- **Hojas de Estilo (QSS incrustado):** Se aplicaron estilos visuales nativos directamente en el XML para estilizar botones con efectos dinámicos al pasar el cursor (hover), bordes sutiles en los campos de entrada y contenedores (QGroupBox) con títulos limpios.
- **Optimización Gráfica:** Se añadió por defecto la propiedad Antialiasing al QGraphicsView para que las líneas conectoras y los nodos circulares del árbol se rendericen con alta definición y sin bordes pixelados.
- **Distribución Escalable:** Los campos de texto de las conversiones y el área gráfica del árbol se encuentran organizados en bloques perfectamente alineados que se ajustan dinámicamente si cambias el tamaño de la ventana.